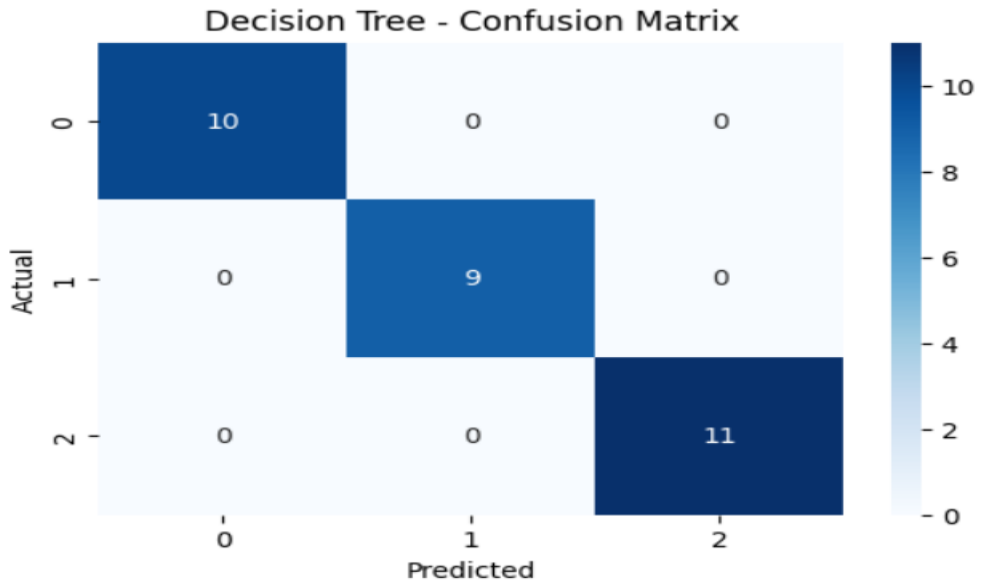
EXP 11

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Decision Tree Classifier")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred),
            annot=True, cmap="Blues", fmt="d")
plt.title("Decision Tree - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

OUTPUT

```
Decision Tree Classifier
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

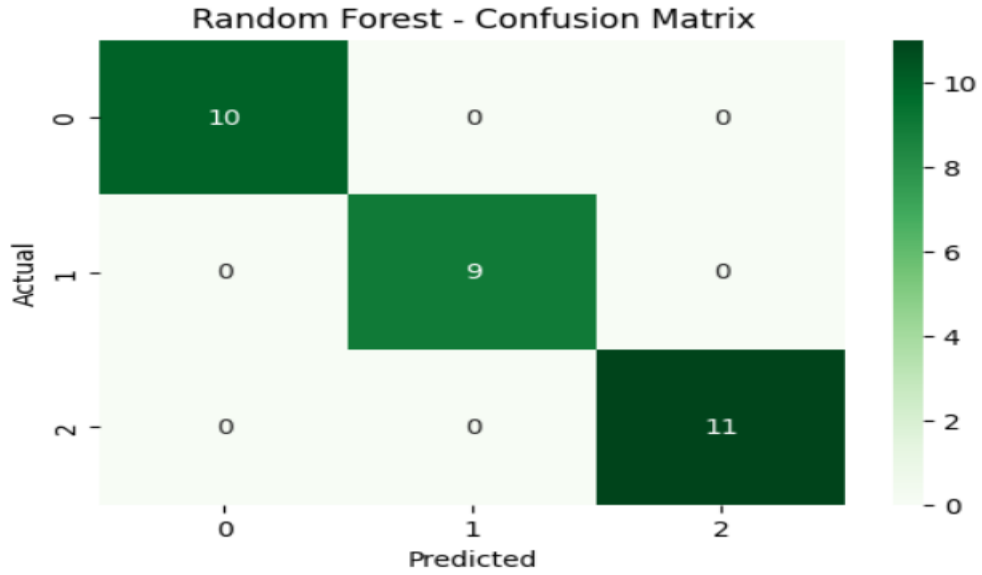


EXP 12

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
iris = load_iris()
X, y = iris.data, iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Random Forest Classifier")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred),
            annot=True, cmap="Greens", fmt="d")
plt.title("Random Forest - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

OUTPUT

```
Random Forest Classifier
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

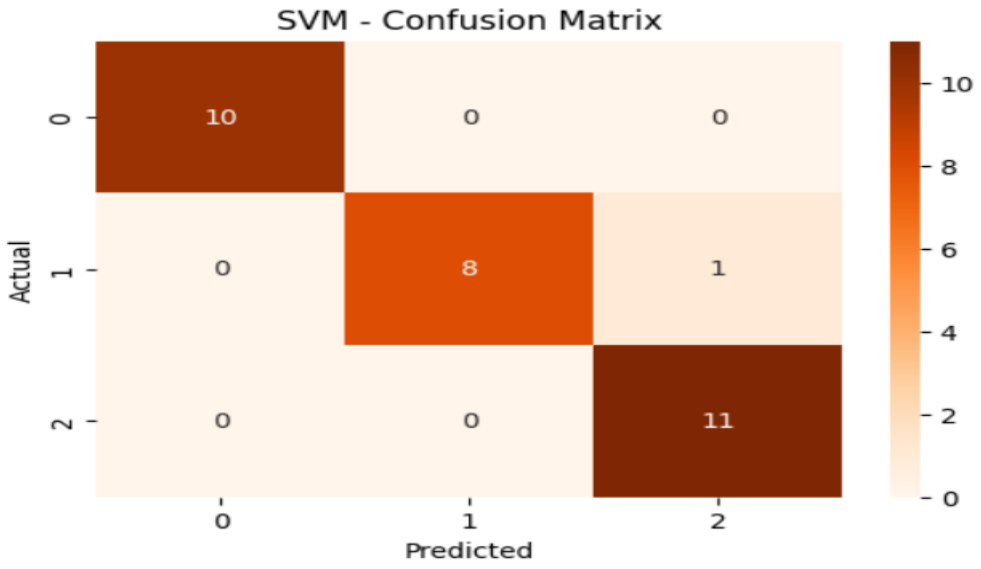


EXP 13

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
iris = load_iris()
X, y = iris.data, iris.target
X = StandardScaler().fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = SVC(kernel="linear")
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Support Vector Machine")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
plt.figure(figsize=(6, 4))
sns.heatmap(confusion_matrix(y_test, y_pred),
            annot=True, cmap="Oranges", fmt="d")
plt.title("SVM - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

OUTPUT

```
Support Vector Machine
Accuracy: 0.9666666666666667
Confusion Matrix:
 [[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]
```

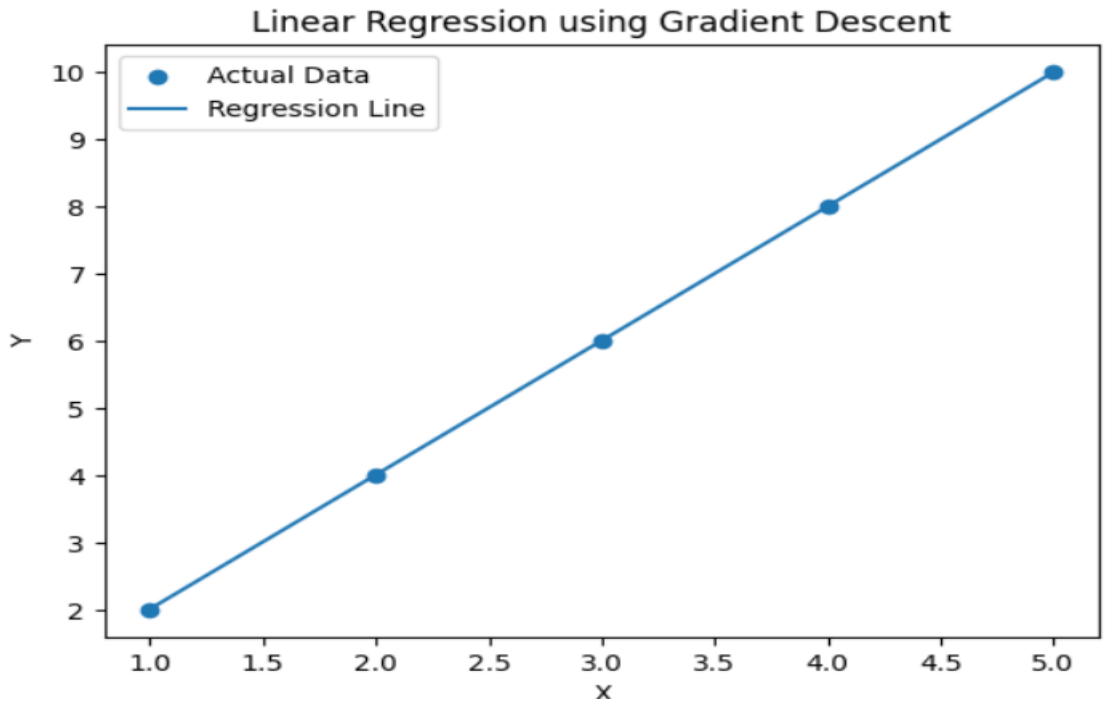


EXP 14

```
import numpy as np
import matplotlib.pyplot as plt
X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])
m = 0
b = 0
lr = 0.01
epochs = 1000
n = len(X)
for i in range(epochs):
    y_pred = m * X + b
    dm = (-2/n) * np.sum(X * (y - y_pred))
    db = (-2/n) * np.sum(y - y_pred)
    m -= lr * dm
    b -= lr * db
print("Slope:", round(m, 2))
print("Intercept:", round(b, 2))
print("Equation: y =", round(m, 2), "x +", round(b, 2))
plt.scatter(X, y, label="Actual Data")
plt.plot(X, m * X + b, label="Regression Line")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression using Gradient Descent")
plt.legend()
plt.show()
```

OUTPUT

```
Slope: 2.0
Intercept: 0.02
Equation: y = 2.0 x + 0.02
```



EXP 15

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
img = np.zeros((300, 300, 3), dtype=np.uint8)
img[:150, :150] = [255, 0, 0]
img[:150, 150:] = [0, 255, 0]
img[150:, :150] = [0, 0, 255]
img[150:, 150:] = [255, 255, 0]
pixels = img.reshape((-1, 3))
pixels = np.float32(pixels)
K = 4
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
            100, 0.2)
compactness, labels, centers = cv2.kmeans(
    pixels, K, None, criteria, 10,
    cv2.KMEANS_RANDOM_CENTERS
)
centers = np.uint8(centers)
segmented = centers[labels.flatten()]
segmented = segmented.reshape(img.shape)
print("Image Segmentation Completed")
print("Number of Clusters:", K)
print("Cluster Centers:")
print(centers)
plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
```

```
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(segmented, cv2.COLOR_BGR2RGB))
plt.title("K-Means Segmented Image")
plt.axis("off")
plt.show()
```

OUTPUT:

```
Image Segmentation Completed
Number of Clusters: 4
Cluster Centers:
[[  0   0 255]
 [255 255   0]
 [255   0   0]
 [  0 255   0]]
```